University of Tabriz

Large Scale Systems Architecture

Designing, Building and Operating at Scale



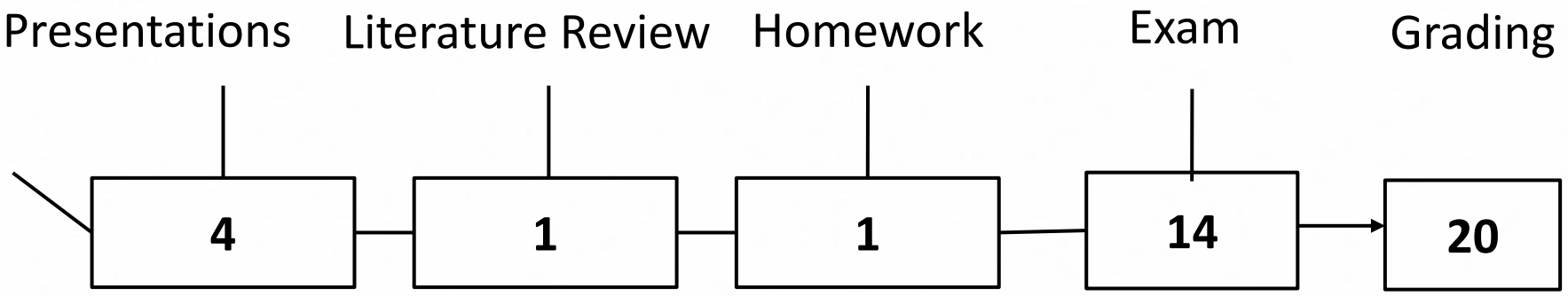
Dr. Ahmad Taghinezhad

Mail:

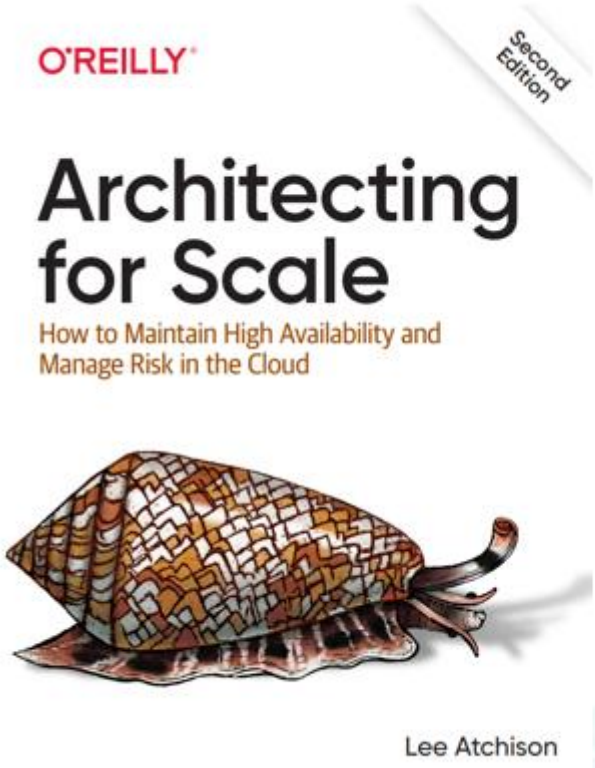
a0taghinezhad@gmail.com

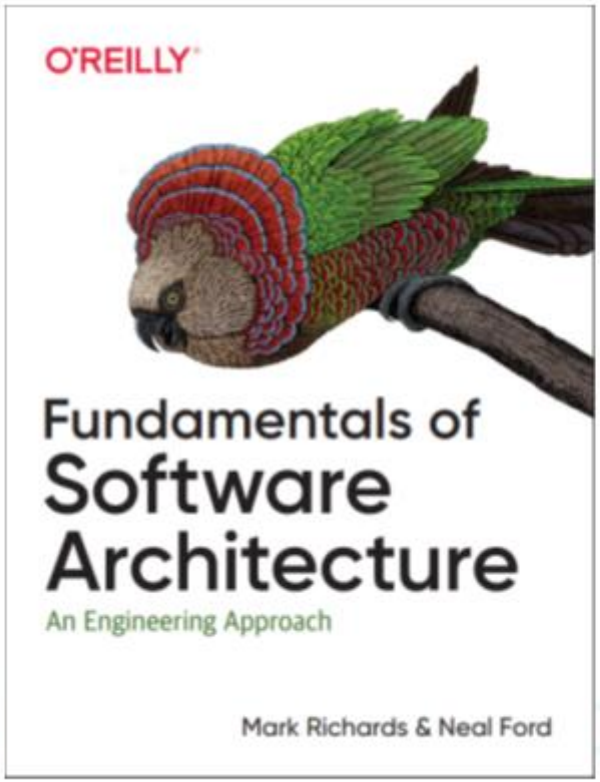
ataghinezhad@tabrizu.ac.ir | 0000-0003-2353-9335 | Google Scholar

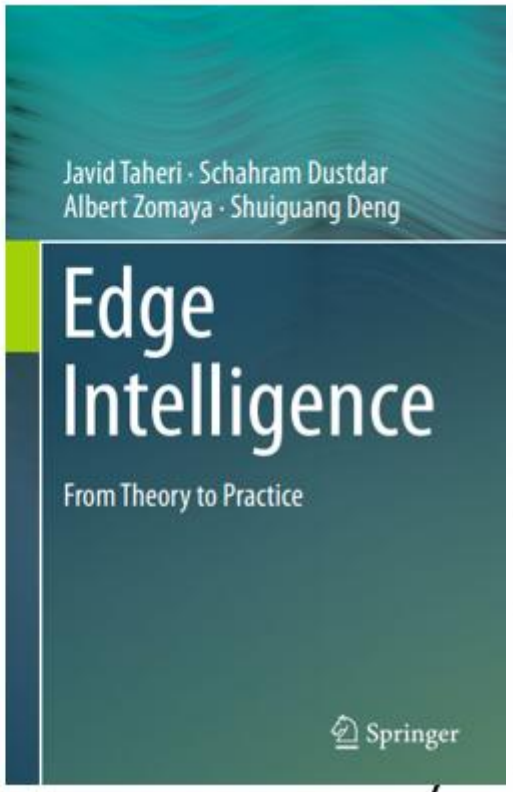
About This Course



References


O'REILLY
Second Edition
Architecting for Scale
How to Maintain High Availability and Manage Risk in the Cloud
Lee Atchison


O'REILLY
Fundamentals of Software Architecture
An Engineering Approach
Mark Richards & Neal Ford


Javid Taheri · Schahram Dustdar
Albert Zomaya · Shuiguang Deng
Edge Intelligence
From Theory to Practice
Springer

Projects include:

Each student must **review at least two papers** and write a **literature review** on these subjects. The subjects are as follows, but not limited to them:

Research theme	Example research topics
Distributed computing	Distributed execution, MapReduce, consensus
Cloud architecture	Cloud-native systems, resource orchestration
Edge computing	Fog computing, edge intelligence
Reliability engineering	Fault tolerance, replication, recovery
Performance engineering	Scheduling, load balancing, scalability
Distributed data	Consistency, partitioning, distributed transactions

Outlines of this course

- Availability: Maintaining Availability in Modern Applications



From Quality Attributes to Operable Architecture

- **Architecture is:**
  - The set of structures needed to reason about a system.
  - The set of design decisions that are costly to change.
  - A balance of quality attributes under constraints.
- **Quality attributes relevant to large-scale systems:**
  - Availability, reliability, scalability, elasticity, performance, security, maintainability, observability, cost.

Availability Is a Business and Architectural Imperative

- **Title:** Availability Is a Business and Architectural Imperative
- **Downtime causes:**
  - Customer frustration and abandonment.
  - Revenue loss.
  - Reputation damage.
  - SLA penalties and contractual risk.
  - Cascading failures across dependent services.
- **Quantitative framing:**
  - 95% monthly availability ≈ **36.5 hours** downtime per month.
  - 99.9% monthly availability ≈ **43 minutes**.
  - 99.999% monthly availability ≈ **26 seconds**.
- **Critical question:** What availability target is justified by the business cost of downtime and the engineering cost of achieving it?



Failure Terminology and Reliability vs. Availability

- Fault, Error, Failure; Reliability vs. Availability
- **Failure chain:**
  - **Fault:** defect or anomalous condition.
  - **Error:** incorrect system state caused by a fault.
  - **Failure:** observable deviation from required service.
- **Failure models:**
  - Crash-stop, crash-recovery, omission, timing, response, and Byzantine failures.
  - Transient vs. permanent vs. intermittent failures.
  - Independent vs. correlated failures.
  - Cascading failures in dependency graphs.



Failure Models

- **Crash-stop** — a process halts and never resumes.
- **Crash-recovery** — a process halts but may restart and rejoin (possibly with lost in-memory state).
- **Omission** — a process fails to send or receive a message it was supposed to.
- **Timing** — a message or action arrives, but too late (or too early) to be useful.
- Example: A stock-trading order arrives after the market price has already moved — technically delivered, but useless.
- **Response (value)** — the system responds, but with an incorrect value.
- Example: A sensor reports temperature 25° C when it is actually 45° C due to a calibration fault.
- **Byzantine** — a component behaves arbitrarily or maliciously, possibly sending conflicting information to different peers.
  - Example: A compromised replica tells one client "balance = $100" and another "balance = $1000." Requires Byzantine fault-tolerant protocols (e.g., PBFT) to tolerate.

Transient vs. Permanent vs. Intermittent Failures

- **Transient** — a one-time failure that disappears without repair.
- Example: A packet is lost due to a momentary Wi-Fi glitch; a retry succeeds. The system self-heals.
- **Permanent** — a failure that persists until the component is repaired or replaced.
- Example: A hard disk develops bad sectors and stops responding entirely; it must be swapped out.
- **Intermittent** — a failure that occurs sporadically, often with no clear pattern.
  - Example: A loose RAM stick causes random bit flips every few hours — hard to reproduce and diagnose.
  - Architectural implication: Transient faults → retries with backoff. Permanent faults → failover/replacement. Intermittent faults → logging, correlation, and replacement of suspect hardware.

The Importance of System Availability for Large-Scale Systems

- System availability is not just a **technical detail**;
  - it's a **fundamental business imperative** for large-scale systems.
- **Fundamental Questions** for All Companies:
  - * Why Buy From You? (Highlight the direct connection between availability and customer acquisition)
  - * What Do Your Customers Think? (Emphasize the impact of downtime on customer satisfaction and brand perception)
  - * How Do You Make Customers Happy? (Connect availability to business goals, revenue, and customer retention)



Defining Availability and Reliability

- **Reliability:**
  - The ability of a system to perform its intended operations **correctly** and **consistently** without errors.
  - **Example:** Software that always returns the correct answer to 2 + 3.
- **Availability:**
  - The ability of a system to be **operational** and **responsive** when needed.
  - **Example:** A system that **consistently returns an answer**, even if the answer is **sometimes** incorrect.
- A system can't be available if it's not reliable. (A system constantly returning wrong answers isn't useful).
- A system can't be reliable if it's not available. (A system that crashes frequently can't be relied upon).
- **Focus:** This course will emphasize building **highly available systems**, assuming your systems are reliable.



Reliability: Testing and Quality; Availability: Architecting for Resilience

- Reliability is often ensured through **thorough testing**.
  - **Test Suites:** We use test suites to catch software bugs and ensure that the system produces the correct output.
  - **Example:** A test suite for adding numbers would verify that 2 + 3 consistently results in 5.
- Availability is more challenging to achieve than reliability.
  - **Network Issues:** Even a reliable application can become unavailable due to network problems.
  - **This Course Focus:** We'll explore strategies to ensure your systems remain available despite network issues, hardware failures, and other challenges.



Causes of Poor Availability at Scale

- **Resource exhaustion:** traffic, memory, connections, file descriptors, queues.
- **Unplanned load-based changes:** rapid growth forces risky changes.
- **Increased moving parts:** more services, developers, features, and interactions.
- **Outside dependencies:** SaaS, cloud services, third-party APIs.
- **Technical debt:** unresolved defects, fragile configurations, undocumented changes.
- **Cascading failures:** one dependency failure propagates.
- **Correlated failures:** shared infrastructure, shared credentials, shared network.
- **Configuration drift:** environments diverge silently.
- **Capacity mismatch:** static capacity against dynamic demand.



Measuring Availability

- Measuring availability is crucial for maintaining a highly available system.
  - By tracking availability, we can gain insights into our application's current performance and observe how it evolves over time.
- The most common method for assessing the availability of a system/application is to calculate the percentage of time it remains accessible to users.
  - This can be expressed using the following formula for a given period:
  - system availability

$$= \frac{\textbf{Total Seconds in Period} - \textbf{Seconds System is Down}}{\textbf{Total Seconds in Period}}$$

Measurement sources:

Source	In one line	Example
Server monitoring	Is the machine healthy?	Server-03 unresponsive 8 min
Synthetic probes	Can an outside user reach us?	/checkout/ probe fails
Application performance monitoring. APM	Does our code return errors?	4% of orders → HTTP 500
Load Balancer / API gateway logs	What fraction of real requests succeeded?	12k/1M requests → 502
Distributed traces	Where exactly did the request fail?	Inventory service timeout
Support incidents	What do users say is broken?	"Cart is empty after login"

Example Calculation

- Let's consider a practical example. Suppose your website experienced two outages during April. The first outage lasted 37 minutes, and the second lasted 15 minutes. How much is the availability in Month?
- 1. **Calculate Total Downtime:**
  - - Total downtime = (37 minutes + 15 minutes) × 60 seconds/minute = 3,120 seconds
- 2. **Calculate Total Seconds in the Month:**
  - - Total seconds in April = 30 days × 86,400 seconds/day = 2,592,000 seconds
- 3. **Calculate Site Availability Percentage:**
  - - Using our formula:
  - Site Availability Percentage = 2,592,000s - 3,120s / 2,592,000s
  - - Site Availability Percentage = ≈ 99.8795%
- Even a small amount of downtime can significantly impact your overall availability percentage.
- Regularly measuring and analyzing availability is essential for ensuring that your application meets user expectations and remains reliable over time.

From Measurement to Service Objectives

Term	Meaning	Example
SLI	**Service Level Indicator: measured signal**	**99.95% successful requests**
SLO	**Service Level Objective: internal target**	**99.9% monthly availability**
SLA	**Service Level Agreement: contractual commitment**	**99.9% with penalties**
Error budget	**Allowable unreliability = 1 − SLO**	**0.1% of monthly time**

Burn rate: How quickly the error budget is being consumed relative to the SLO window. High burn rate should trigger alerts before the budget is exhausted.

Critical question: Should planned maintenance consume the error budget? Internally, yes — because users still experience downtime. An SLA may exclude it, but user experience does not.

Burn rate

- **Burn rate** = how fast you are spending your error budget compared to the "normal" rate for the SLO window.
- **Normal rate = 1×** means you will exactly use up the budget at the end of the window.
- $\text{Burn rate} = \dfrac{\text{error rate observed}}{\text{error budget rate allowed}}$

  - **SLO:** 99.9% monthly availability → **error budget = 0.1%** of the month.
  - In a 30-day month (43,200 minutes), the budget = **43.2 minutes** of allowed downtime.
  - Normal spending rate = 43.2 min / 30 days ≈ **1.44 min/day**.

Why it matters: Burn rate detects *trends*, not just absolute downtime. A slow leak (e.g., 2× every day) exhausts the budget just as surely as one big outage — but without burn-rate alerting you'd only notice at month-end when it's too late to react.

The Nines and Downtime Budgets

"The Nines" as Availability Shorthand

Example analysis:
99.8795% falls short of 3 nines (99.9%). Achieving 5 nines requires redundancy, automated failover, strong observability, and rigorous change management.

Instructor note: Each additional nine usually increases architectural and operational cost nonlinearly. Ask students to identify the business case for each target.

Nines	Percentage	Approx. monthly downtime
2 nines	99%	432 minutes
3 nines	99.9%	43 minutes
4 nines	99.99%	4 minutes
5 nines	99.999%	26 seconds
6 nines	99.9999%	2.6 seconds

Example Analysis

- In your earlier example, your website achieved an availability of approximately 99.8795%, which falls short of **the 3 nines (99.9%) standard**.
  - This means that while your site is relatively reliable, it **does not meet** the expectations typically associated with **high-availability applications**.
- For a website aiming for 5 nines of availability, the requirement is extremely stringent: only 26 seconds of monthly downtime.
  - Achieving this level may necessitate advanced redundancy, failover mechanisms, and robust monitoring systems.

Determining Your Availability Goals

- Determining what constitutes a reasonable availability number depends on various factors:
  - 1. **System Type**: Critical applications (e.g., financial services) may require higher availability compared to less critical sites (e.g., personal blogs).
  - 2. **Customer Expectations**: Understand what your users expect in terms of uptime and reliability.
  - 3. **Business Needs**: Consider the impact of downtime on your revenue and brand reputation.
  - 4. **Industry Standards**: Research what competitors or similar businesses are achieving.

Determining Your Availability Goals

- Common Thresholds

  - For many **basic** web applications**, 3 nines** (99.9%) is often considered acceptable, allowing for about 43 minutes of downtime each month.
  - More **critical** applications may aim for **4 nines** (99.99%), while mission-critical systems might target 5 nines (99.999%) or even higher.

Understanding the Impact of Planned Outages on Availability

- Planned outages, while necessary for maintenance and updates, directly affect the overall availability of an application.
  - Many organizations mistakenly believe that regular **maintenance** windows do not **count against their availability metrics**. However, as illustrated in your example, they certainly do.

Ex

Example Calculation

- Here's a comment that I often overhear: "Our application never fails. That's because we regularly perform system maintenance. We keep our availability high by scheduling **weekly two-hour maintenance** windows and performing maintenance during these windows."
- Let's break down the example provided:
  - 1. Total Hours in a Week:
    - - 7 days × 24 hours = 168 hours
  - 2. Hours Unavailable Each Week:
    - - 2 hours (for planned maintenance)
  - 3. Calculating Availability:
    - - Availability Formula:
    - Availability = Total Hours in Period - Hours System is Down/Total Hours in Period
    - - Plugging in Values:
    - Availability = 168 hours - 2 hours/168 hours = 166/168≈ 0.9881 or 98.81%
- Without having a single failure of its application, the best this organization can achieve is 98.8% availability. This falls short of even 2 nines availability (98.8% versus 99%).

Implications for User Experience

- - **Negative User Experience**: Regardless of whether downtime is planned or unplanned if customers cannot access the application when they need it, their experience is negatively impacted.
- - Expectation Management: Customers expect high availability, and any downtime—planned or not—can lead to frustration, loss of trust, and potentially lost revenue.

Strategies to Mitigate Impact

- 1. **Schedule During Off-Peak Hours**: If possible, schedule maintenance during times when user activity is lowest to minimize impact.
- 2. **Communicate Clearly**: Inform users well in advance about scheduled maintenance windows and the expected downtime.
- 3. **Implement Redundancy**: Use load balancing and failover systems to allow for maintenance without affecting availability.
- 4. **Automate Maintenance Tasks**: Where feasible, automate maintenance tasks to reduce the duration of downtime.
- 5. **Monitor and Analyze**: Continuously monitor application performance and user feedback to adjust maintenance practices as necessary.

Improving Your Availability When It Slips

- Your application is operational and online.
  - Systems are in place, your team is efficient, and everything seems to be going well.
  - Traffic is increasing, and your sales team is pleased.
    - an unexpected outage occurs. Initially, it feels manageable—after all, your availability has been fantastic until now.
    - But then it happens again. And again.
- As outages accumulate, concerns grow. Your CEO starts to worry, customers ask questions, and your sales team becomes anxious. What was once a stable system is now facing instability, and the situation demands attention.

What to Do When Availability Begins to Slip

- **1. Measure and establish a baseline.** Track uptime/downtime, calculate availability over time, and correlate it with key events to spot trends early.
- **2. Prioritize with service tiers.** Classify services by business criticality (e.g., checkout vs. reviews) so resources go where impact is highest. *Example:* Checkout gets dedicated servers; reviews can go down without losing sales.
- **3. Make risk visible.** Maintain a risk matrix of failure points and technical debt; review it regularly and act on mitigations.
- *Example:* Risk matrix flags "single database = high risk" → team adds a read replica.
- **4. Automate what is manual.** Automate processes and deployments for consistency, speed, and fewer human errors.
- **5. Control change safely.** Keep configurations under centralized management, and enable fast rollback for low-risk experimentation.
- **6. Improve continuously.** Regularly assess systems and practices, and keep availability a visible core priority as the system evolves.

Risk Matrix

CONSEQUENCE →

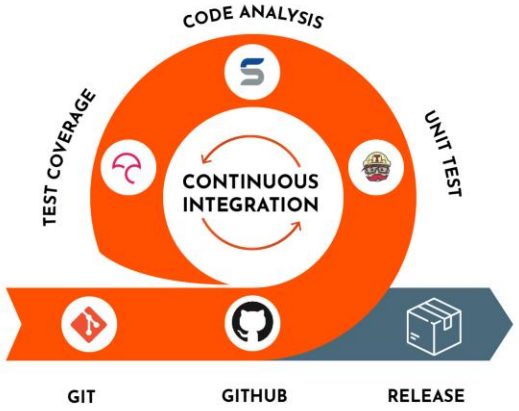
LIKELIHOOD	1. Insignificant No material service impact	2. Minor Limited user impact; workaround available	3. Moderate Degraded service; recovery required	4. Major Extended outage or major business impact	5. Catastrophic Data loss, severe breach, or platform-wide failure
A. Almost certain Expected in most operating conditions	**MEDIUM | 8** Brief API latency spike	**HIGH | 16** Payment service outage	**HIGH | 18** Regional service outage	**CRITICAL | 23** Permanent loss of customer data	**CRITICAL | 25** Platform-wide destructive failure
B. Likely Occurs frequently	**MEDIUM | 7** Repeated login failures	**MEDIUM | 10** Queue backlog under load	**HIGH | 17** Core service unavailable	**HIGH | 20** Multi-region failover failure	**CRITICAL | 24** Critical data corruption
C. Possible May occur sometimes	**LOW | 3** Occasional UI defect	**MEDIUM | 9** Search index inconsistency	**MEDIUM | 12** Peak-hour performance drop	**HIGH | 19** Delayed order processing	**HIGH | 22** Sensitive-data exposure
D. Unlikely Not expected, but plausible	**LOW | 2** Rare noncritical bug	**LOW | 5** Intermittent cache issue	**MEDIUM | 11** Database replica lag	**MEDIUM | 14** Deployment rollback delay	**HIGH | 21** Single-region outage
E. Rare Only in exceptional conditions	**LOW | 1** Cosmetic display issue	**LOW | 4** Rare report formatting error	**LOW | 6** Occasional notification delay	**MEDIUM | 13** Minor analytics delay	**MEDIUM | 15** Low-impact background-job failure

LOW MEDIUM HIGH CRITICAL

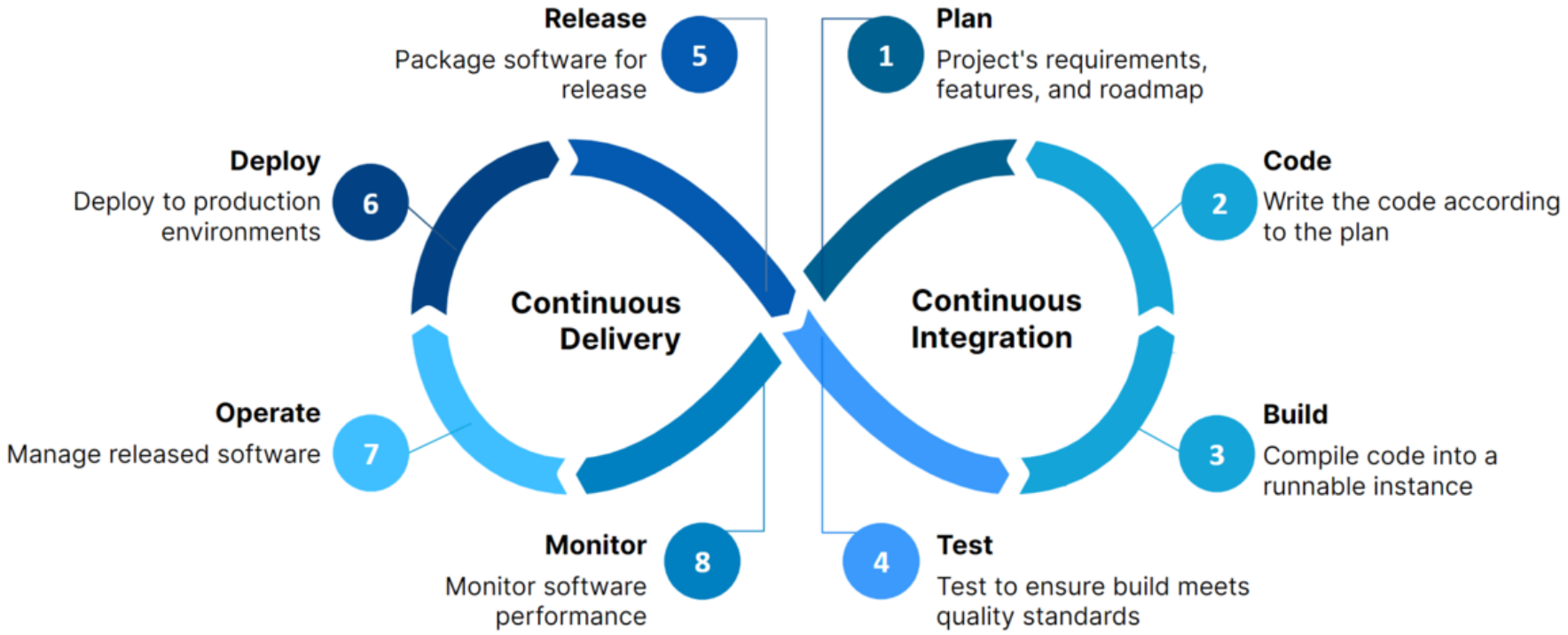
Example risks are illustrative; assess each risk using your system context, controls, and business impact.

CI/CD

- **What it is:** Developers merge code into a shared repository frequently — several times a day.
- **Why it matters:** Integration problems surface early, when they are cheap to fix, instead of piling up into painful late-stage conflicts.
- **Example:** Three developers push code to the same GitHub repo. On every push, the CI system automatically builds the app and runs the test suite. If one developer's change breaks a test, the team is notified within minutes and fixes it before it reaches production.



CI/CD


Release
Package software for release
5
Plan
1
Project's requirements, features, and roadmap
Deploy
Deploy to production environments
6
Code
2
Write the code according to the plan
Continuous Delivery
Continuous Integration
Operate
Manage released software
7
Build
3
Compile code into a runnable instance
Monitor
Monitor software performance
8
4
Test
Test to ensure build meets quality standards

CI/CD

1. **Continuous Deployment (CD)**:
    1. **What it is**: CD extends CI by automating the deployment process. It ensures that code changes are automatically deployed to production environments once they pass all tests.
    2. **Why it matters**: CD reduces manual intervention, minimizes the time between writing code and deploying it, and increases the reliability of releases.
    3. **Example**: Continuing with our web application scenario, after successful CI, the CD pipeline takes over. It automatically deploys the tested code to staging or production servers. Users get access to new features or bug fixes without waiting for a manual deployment process.
2. **The CI/CD Pipeline**:
    1. The CI/CD pipeline is the set of automated steps that code goes through from development to production.

What is CI/CD?

CI/CD = Continuous Integration + Continuous Deployment

🎯 **Goal:** Deliver high-quality code faster and safely.

Term	Meaning	Example
CI	Automatically test and check code whenever it changes	Run tests when you push to GitHub
CD	Automatically deploy the tested code to production	Update your website automatically

CI/CD Workflow Overview

CI/CD Workflow Overview

From Code to Cloud — Automatically!

1 Developer writes code



2 Pushes to GitHub

Push successful!

3 GitHub Actions runs tests

GitHub Actions

- pytest ... Passed
- flake8 ... Passed

4 If all tests pass → Build Docker image

docker

Image built!

5 Deploy app to cloud server



Deployed successfully!

Tools at a Glance

Tool		Purpose
	Git & GitHub	Store and manage your code
	GitHub Actions	Runs automated CI/CD pipelines
	pytest	Runs tests automatically
black	flake8 / black	Check and format your code
	Docker	Package your app for deployment
aws	AWS / Heroku / Render	Host your app online

Project Structure Example

```
my_python_app/
├── .github/workflows/ci-cd.yml   # CI/CD workflow
├── app/                          # Application code
│   └── main.py
├── requirements.txt              # Dependencies
├── Dockerfile                    # Build Docker image
└── tests/
    └── test_main.py              # Tests
```



Tools Used in Python CI/CD

Tool	Purpose
Git & GitHub	Store and manage your code
GitHub Actions	Runs automated CI/CD pipelines
pytest	Runs tests automatically
flake8 / black	Check and format your code
Docker	Package your app for deployment
AWS / Heroku / Render	Host your app online



CI Example – Testing and Linting

.github/workflows/ci-cd.yml

```
name: Python CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10"
      - run: pip install -r
requirements.txt pytest flake8
      - run: flake8 app
      - run: pytest
```

CD Example – Deploying with Docker

After tests pass:

```
deploy:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKER_USER }}
        password: ${{ secrets.DOCKER_TOKEN }}
    - uses: docker/build-push-action@v6
      with:
        push: true
        tags: yourname/my_python_app:latest
```

Dockerfile Example

Dockerefile:

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app/main.py"]
```

Packages your code so it runs the same anywhere — local or cloud.

Dockerfile Example

Step	Stage	Tool
1	Write code	VSCode / PyCharm
2	Push to GitHub	Git
3	Test automatically	GitHub Actions + pytest
4	Build container	Docker
5	Deploy online	AWS / Render / Heroku



Example: Database Schema Changes:Avoid Manual Operations in Production

- **1. Testing Before Deployment**
  - **Approach:** Generate migrations with makemigrations and apply them in a staging environment using migrate.
  - **Benefit:** Ensures migrations run safely before reaching production.
- **2:** Define fields with defaults or constraints in Django models.
  - **Ben. Customizable & Controlled**
  - **Approachefit:** Keeps schema aligned with application and business logic.
- **3. Peer Review**
  - **Approach:** Submit migration files in pull requests for review.
  - **Benefit:** Detects logic or performance issues early.
- **4. Version Control**
  - **Approach:** Commit all migration files to Git.
  - **Benefit:** Tracks who made schema changes and why.



Example: Database Schema Changes:Avoid Manual Operations in Production

- **5. Environment Consistency**
  - **Approach:** Use Django's migration system across dev, staging, and production.
  - **Benefit:** Keeps all environments in sync with identical schemas.
- **6. Operational Stability**
  - **Approach:** Apply migrations via Django commands, not manual SQL.
  - **Benefit:** Prevents configuration drift and supports rollback with migrate <previous>.
- **7. Auditability**
  - **Approach:** Use Django's django_migrations table and deployment logs.
  - **Benefit:** Enables traceability and impact analysis of each change.

Automated Deployments

- Consistency: Automating deployments ensures that changes are applied uniformly across your system. This means you can confidently apply similar changes later with predictable results.

- - Reliable Rollbacks: Automated deployment systems enhance the reliability of rolling back to known good states, minimizing downtime and risk

Maintain Configuration Management:

- Manually modifying server configuration (e.g., editing files, tweaking kernel variables, changing environment settings) is risky and non-repeatable.
  Instead, use **scripts** or **declarative configuration files** under version control.
- Example: YAML-based Configuration Automation
  - YAML file defines all configuration parameters and the target servers that should receive them.

configs/app_config.yaml:

```
app:
  max_open_files: 65536
  env: production
  log_level: INFO
targets:
  - host: app-01.example.internal
    user: ubuntu
  - host: app-02.example.internal
    user: ubuntu
```

"We want to connect to two Ubuntu servers (`app-01` and `app-02`), and set their open-file limit to 65,536 for our production environment."

Maintain Configuration Management:

- **2. Uniformity Across Servers**

By automating configuration updates via apply_config.py, every server receives **identical configuration parameters**.

🔁 Example: When adding a new Django app server, simply include its IP in the YAML file ; no manual setup required.

Infrastructure as Code (IaC)

- **Infrastructure as Code** means defining infrastructure (servers, load balancers, networks, etc.) in a **machine-readable format**, typically managed with tools like **Terraform**, **Ansible**, **Puppet**, or **C**

Python script for managing configurations centrally in **AWS Systems:**

This pushes all configuration variables to AWS Parameter Store.
Your Django app can fetch these at runtime, ensuring all instances use the same configuration values.

```
# put_params.py
import boto3
import yaml

ssm = boto3.client("ssm", region_name="us-east-1")
cfg = yaml.safe_load(open("configs/app_config.yaml"))
params = cfg["app"]

for k, v in params.items():
    name = f"/myapp/{cfg['app']['env']}/{k}"
    ssm.put_parameter(Name=name, Value=str(v), Type="String",
Overwrite=True)
    print("Updated", name)
```

Comprehensive Application

- **Real-World Pitfall Example**

“We had an issue last night. I tweaked the kernel variable and increased the number of open files manually.”

This may temporarily fix a problem, but:

Other servers remain misconfigured.

The change isn’t documented or reproducible.

It creates inconsistency and risk.

✅ **Better:** Update configs/app_config.yaml and rerun apply_config.py.
The change is applied uniformly, logged, and versioned.

Five Focus Areas to Improve Application Availability

- Building a scalable application with high availability is challenging. Issues can arise unexpectedly, causing disruptions for users. Here are five key focuses to ensure your application remains available:

Five Focus Areas to Improve Application Availability

- 1. Build with Failure in Mind
  - - Anticipate Failures: As Werner Vogels, CTO of Amazon, states, “Everything fails all the time.”
  - **1) Design Considerations:**
    - - Error Handling: Implement error-catching mechanisms.
    - - Example:
      - Use **retry logic** to attempt to reconnect to a failed service before giving up.
  - **2) Dependencies:**
    - Circuit breaker patterns are particularly useful for handling dependency failures because they can reduce the impact a dependency failure has on your system. Without a circuit breaker, you can decrease the performance of your application because of a dependency failure
  - **3) Customers:** What do you do when a component that is a customer of your system behaves poorly

Five Focus Areas to Improve Application Availability

- **2) Always Think About Scaling**
  - Prepare for Growth: Design your system to handle increased loads without performance degradation.
    - New server must be added to your system easily.
    - - Example: Use load balancers to distribute traffic evenly across servers.
  - Reload only dynamic parts and use (CDN) and browser from static parts. Don’t load the entire website from sever every time.

Five Focus Areas to Improve Application Availability

- **3) Mitigate Risk**
  - - Identify Vulnerabilities: Assess components that could fail and develop strategies to minimize their impact.
  - - Example: Utilize redundancy for critical services (e.g., multiple database replicas).
    - Imagine you are selling a product on a website, and your search system is a different system, if it fails, all your system fails. How would you respond to that?
      - One way is to show your most popular product and respond to apologize for the search problem.

Five Focus Areas to Improve Application Availability

- 4) **Monitor Availability:** You can't know if there is a problem in your application unless you can see a problem
  - **Server monitoring** To monitor the health of your servers and make sure they keep operating efficiently.
  - **Configuration change monitoring:** To monitor your system configuration and identify if and when changes to your infrastructure impact your application.
  - **Application performance monitoring:** To look inside your application and services to make sure they are operating as expected.
  - **Synthetic testing:** To examine in real time how your application is functioning from the perspective of your users, in order to catch problems customers might see before they actually see them.
  - **Alerting:** To inform appropriate personnel when a problem occurs so that it can be quickly and efficiently resolved, minimizing the impact on your customers.

Five Focus Areas to Improve Application Availability

- 4) **Monitor Availability:** You can't know if there is a problem in your application unless you can see a problem

a) **Server Monitoring**

   a) **Purpose:** Check system health (CPU, memory) regularly.
   Example: Uses psutil to read resource usage and sends alerts if thresholds are exceeded.
   Benefit: Detects overload or performance degradation early.

b) **Configuration Change Monitoring**

   Purpose: Track if key configuration files (e.g., YAML specs) are modified.
   Example: Computes a SHA-256 hash of the file and compares it to the last stored value.
   Benefit: Prevents silent configuration drifts that could break consistency.

c) **Application Performance Monitoring (APM)**

   Purpose: Measure request time and capture slow or failing requests.
   Example: Django middleware logs response times and sends alerts for slow requests or exceptions.
   Benefit: Reveals performance bottlenecks and unhandled errors in real time.

Five Focus Areas to Improve Application Availability

- 4) **Monitor Availability:** You can't know if there is a problem in your application unless you can see a problem

d) Synthetic Testing

Purpose: Simulate user actions (e.g., loading /login/, /dashboard/) from outside the system.
Example: Python requests periodically checks endpoints and alerts on non-200 responses.
Benefit: Detects user-facing problems before real users encounter them.

e) Alerting

Purpose: Notify operators instantly when issues occur.
Example: Sends alerts via Slack or email using reusable helper functions.
Benefit: Enables quick response, minimizing downtime or customer impact.

Five Focus Areas to Improve Application Availability

- **5) Respond to Availability Issues Predictably**
  - Unresponsive Services:
    - When a service becomes unresponsive, several remedies can be employed:
      - Run diagnostic tests to identify the issue.
      - Restart a known problematic daemon.
      - Reboot the server if necessary.
  - Standardized Failure Handling:
    - Implement standard procedures to reduce system downtime.
    - These processes aid in identifying the root cause of recurring issues.
    - Service owners must be alerted first to address problems
    - Related teams should also receive alerts to prepare for potential impact on their dependent services.

Five Focus Areas to Improve Application Availability

- Real-World Example: Icon Failure
  - A past project experienced a major outage due to a third-party icon generation system failure. The application assumed this dependency would always function. When it failed, the entire application went down.
- Lessons Learned:
  - - Anticipate Dependency Failures: Implement logic to handle such failures gracefully.
  - - Error Recovery: Detect the failure and either remove the icon or catch the error, allowing the rest of the application to function normally.

End of Chapter 1